

TD 5

Modèles linéaires, optimisation et régularisation

Objectifs du TD

Ce TD a pour objectif de :

- Comprendre les modèles linéaires et log-linéaires
- Manipuler les fonctions softmax et sigmoid
- Comprendre les fonctions de perte
- Étudier l'optimisation par descente de gradient
- Comprendre la régularisation
- Identifier les limites des modèles linéaires

1 Classification linéaire

On considère un classifieur linéaire :

$$f(\vec{x}) = \vec{x} \cdot \vec{w} + b$$

Exercice 1

- Donner l'équation de la frontière de décision associée à ce modèle.
- Expliquer le rôle du vecteur de poids $\vec{w}$ et du biais b .
- Que se passe-t-il si $b = 0$?

2 Classification logistique

La sortie du modèle est transformée par la fonction sigmoid :

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Exercice 2

- Montrer que $\sigma(z) \in (0, 1)$.
- Interpréter la sortie $\hat{y} = \sigma(f(\vec{x}))$ comme une probabilité.
- Pourquoi utilise-t-on la sigmoid plutôt qu'une fonction seuil ?

3 Classification multi-classes

On considère maintenant un problème à K classes avec la fonction softmax :

$$\text{softmax}(\vec{z})_{[i]} = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Exercice 3

- Montrer que les sorties de la softmax sont positives et somment à 1.
- Quelle est la différence entre softmax et sigmoid ?
- Pourquoi la softmax est-elle adaptée à la classification multi-classes ?

4 Fonctions de perte

Exercice 4 : Perte logistique

La perte logistique est définie par :

$$L(y, \hat{y}) = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$$

- Calculer la perte pour $y = 1$ et $\hat{y} = 0.9$, puis $\hat{y} = 0.1$.
- Interpréter les résultats.
- Pourquoi cette perte pénalise fortement les prédictions confiantes mais incorrectes ?

Exercice 5 : Entropie croisée

$$L(\vec{y}, \vec{\hat{y}}) = - \sum_i y_i \log(\hat{y}_i)$$

- Supposer que $\vec{y}$ est un vecteur one-hot. Simplifier l'expression.
- Expliquer pourquoi cette perte est adaptée à la softmax.

5 Optimisation par descente de gradient

Exercice 6

- Expliquer le principe général de la descente de gradient.
- Donner la règle de mise à jour des paramètres.
- Quel est le rôle du taux d'apprentissage η ?

6 Stochastic Gradient Descent

Exercice 7

- Quelle est la différence entre descente de gradient classique et SGD ?
- Pourquoi le SGD introduit-il du bruit ?
- Quel est l'avantage du mini-batch SGD ?

7 Régularisation

Exercice 8

On considère la fonction objectif suivante :

$$\mathcal{L}(\Theta) + \lambda R(\Theta)$$

- Quel est le rôle du terme de régularisation ?
- Expliquer l'effet du paramètre λ .

Exercice 9

- Donner l'expression de la régularisation L_2 .
- Donner l'expression de la régularisation L_1 .
- Comparer leurs effets sur les poids.

8 Limites des modèles linéaires

Exercice 10 : XOR

- Tracer les points du problème XOR.
- Expliquer pourquoi ils ne sont pas linéairement séparables.
- Quelle solution permet de résoudre ce problème ?

Question bonus

Pourquoi les réseaux de neurones peuvent être vus comme une succession de transformations non linéaires suivies de modèles linéaires ?

$$f(g(x)) = A(Cx + d) + b = ACx + (Ad + b) \quad (1)$$

AC is a matrix and $Ad + b$ is a vector, so we see that composing affine maps gives you an affine map.

From this, you can see that if you wanted your neural network to be long chains of affine compositions, that this adds no new power to your model than just doing a single affine map.

If we introduce non-linearities in between the affine layers, this is no longer the case, and we can build much more powerful models.

There are a few core non-linearities. $\tanh(x)$, $\sigma(x)$, $\text{ReLU}(x)$ are the most common. You are probably wondering : "why these functions ? I can think of plenty of other non-linearities." The reason for this is that they have gradients that are easy to compute, and computing gradients is essential for learning. For example

$$\frac{d\sigma}{dx} = \sigma(x)(1 - \sigma(x)) \quad (2)$$